

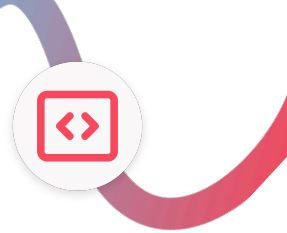
## LEARN THE TERMINAL

- CLICK TO JUMP TO THE PAGE -

|                                           |           |
|-------------------------------------------|-----------|
| <b>1. Terminal 101.....</b>               | <b>2</b>  |
| Terminal.....                             | 2         |
| Command-line.....                         | 2         |
| Shell.....                                | 2         |
| Graphical user Shell.....                 | 2         |
| Command Line Interface (CLI).....         | 3         |
| Relative path.....                        | 3         |
| Absolute path.....                        | 4         |
| Directory.....                            | 4         |
| Working Directory.....                    | 4         |
| Standard input (stdin).....               | 4         |
| Standard output (stdout).....             | 5         |
| Standard error (stderr).....              | 5         |
| <b>2. Location and navigation.....</b>    | <b>6</b>  |
| Print Working Directory.....              | 6         |
| List.....                                 | 6         |
| List -a.....                              | 6         |
| List -l.....                              | 6         |
| List -la.....                             | 6         |
| Change Directory.....                     | 7         |
| Change Directory <relative path>.....     | 7         |
| <b>3. Managing files and folders.....</b> | <b>8</b>  |
| touch.....                                | 8         |
| mkdir.....                                | 8         |
| echo.....                                 | 8         |
| cat.....                                  | 8         |
| cp.....                                   | 8         |
| mv.....                                   | 9         |
| rm.....                                   | 9         |
| rm -r.....                                | 9         |
| rm -f.....                                | 9         |
| rm -rf.....                               | 9         |
| rmdir.....                                | 10        |
| wildcards.....                            | 10        |
| <b>4. Concatenate commands.....</b>       | <b>11</b> |
| Concatenating commands ;.....             | 11        |
| <b>5. CLIs.....</b>                       | <b>12</b> |



|                                                         |           |
|---------------------------------------------------------|-----------|
| Git.....                                                | 12        |
| Node.....                                               | 12        |
| NPM.....                                                | 12        |
| <b>6. Redirecting input and output.....</b>             | <b>13</b> |
| >.....                                                  | 13        |
| >>.....                                                 | 13        |
| <.....                                                  | 13        |
| .....                                                   | 13        |
| Sort.....                                               | 13        |
| Uniq.....                                               | 14        |
| Grep.....                                               | 14        |
| Grep -i.....                                            | 14        |
| Grep -R.....                                            | 14        |
| Grep -RI.....                                           | 14        |
| Sed.....                                                | 15        |
| Sed -i.....                                             | 15        |
| <b>7. Nano editor.....</b>                              | <b>16</b> |
| Nano.....                                               | 16        |
| Saving changings.....                                   | 16        |
| Copy.....                                               | 16        |
| Paste.....                                              | 17        |
| Undo.....                                               | 17        |
| Redo.....                                               | 17        |
| Exiting nano.....                                       | 17        |
| <b>8. Configuring the command line environment.....</b> | <b>18</b> |
| Environment.....                                        | 18        |
| Bash profile.....                                       | 18        |
| Source.....                                             | 18        |
| Alias.....                                              | 18        |
| Environment Variables.....                              | 18        |
| PS1.....                                                | 19        |
| HOME.....                                               | 19        |
| PATH.....                                               | 19        |
| env.....                                                | 19        |



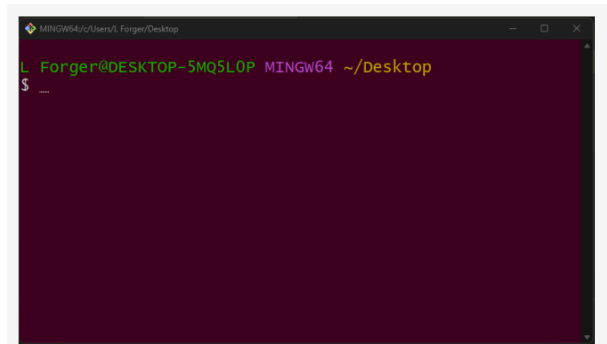
## 1. Terminal 101

### Terminal

A graphical interface that enables access to one or more shells.

The terms **terminal**, **shell** and **command-line** are often used interchangeably. However, it is important to note that the terminal specifically designates the graphical window the user interacts with.

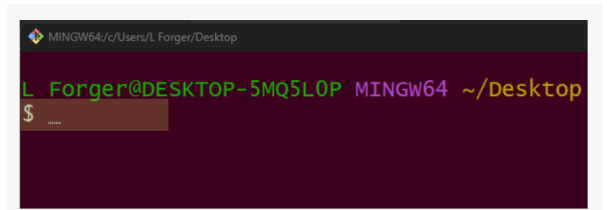
[Back to index](#)



### Command-line

It is the line in the terminal where the commands are inputted. The expression can also be used to designate the command itself.

[Back to index](#)



### Shell

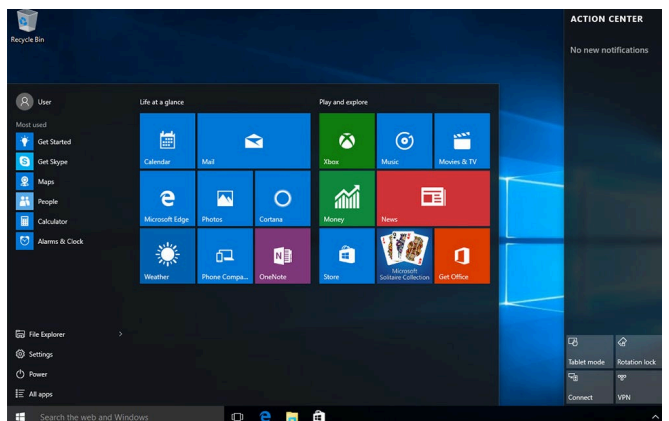
It is a software that exposes the services of an operational system to the user, processes and executes commands and oftentimes returns an output. Most shells also manage foreground and background processes, command history and command-line editing. Shells can use either a **command-line interface** (CLI) or a **graphical user interface** (GUI).

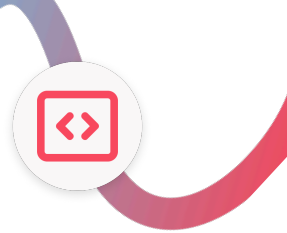
[Back to index](#)

### Graphical user Shell

It is an interface that allows users to interact with electronic devices through graphical icons and visual indicators. Windows File Explorer and MacOS Finder are some examples of GUIs that enable access to core functionality of the operating system.

[Back to index](#)

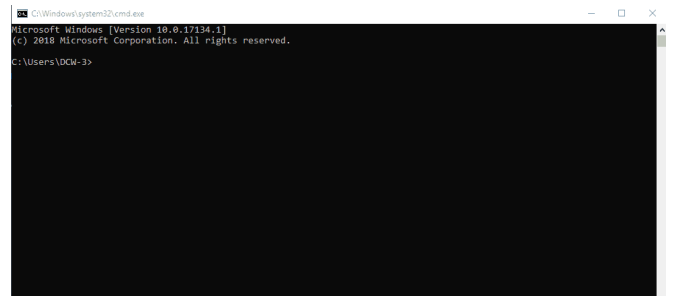




## Command Line Interface (CLI)

A command line interface (CLI) is a text-based interface used to run programs, manage computer files and interact with the computer. The **terminal** can be considered a type of command line interface, and is referred to as such sometimes. Shells come with their own Command Line Interfaces, and often developer softwares like Visual Studio Code and Git add their own set of CLI commands to your computer to allow for interaction via the terminal.

[Back to index](#)



## Relative path

Describes the location of a file or directory in relation to the current file or directory. It can be used to link files in your projects or to switch the current location of the terminal to another one.

To declare a relative path, it is really important to look at the **hierarchy** of the **file system**. This hierarchy works as a tree, being the root folder, the one at the highest level in the hierarchy.

To express those hierarchical relationships, the relative path uses two elements:

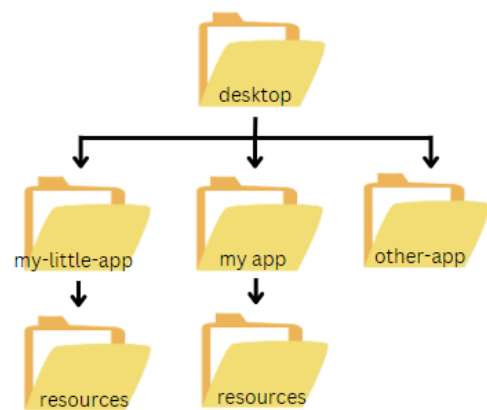
The **'directory-name'** is the name of a **child directory** to the current one.

The **'..'** refers to the parent directory of the current one.

The **'directory-names'** and the **'..'** can be concatenated by using **'/'**.

Names of files and folders with spaces will be between quotation marks.

[Back to index](#)



**desktop** -> relative path to the **desktop** directory, child to the **working directory**

**desktop/my-little-app/resources** -> relative path to the **resources** directory, child to **my-little-app**, grandchild to **desktop** and great-grandchild to the **working directory**

**..** -> relative path to the parent of the **working directory**

**desktop/"my little app"** -> relative path to the **my little app** directory, child to **desktop** directory and grandchild to **working directory**

**../my-little-app** -> relative path to the **my-little-app** directory, child to **desktop** directory and sibling to the **working directory**



### Absolute path

Specifies the location of a file or directory from the root directory. It is given in the terminal when the **pwd** command is entered.

[Back to index](#)

### Directory

It is a file system cataloguing structure which contains references to other computer files, and possibly other directories.

In the image on the right, for example, there are two directories: the Project directory and the Resources directory.

[Back to index](#)

### Working Directory

It is the directory where the terminal is currently located.

If our terminal is located at

**C:/Users/L/Desktop/Project**, that means that our working directory is the project directory.

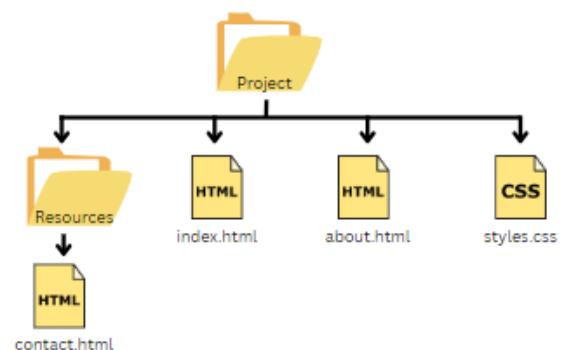
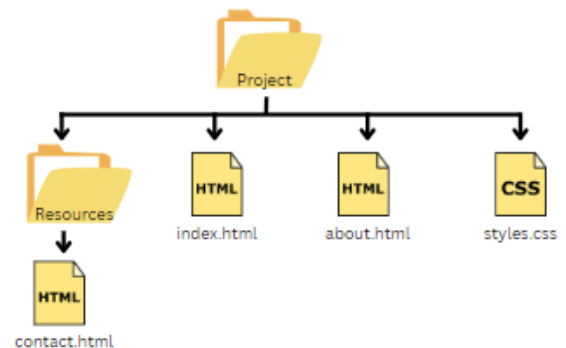
[Back to index](#)

### Standard input (stdin)

Is the information inputted into the terminal.

[Back to index](#)

C:/Users/L/Desktop



**echo "hello" -> hello** is the stdin



### Standard output (stdout)

Is the information outputted after a process is run.

[Back to index](#)

```
PS C:\Users\maria> echo "hello"
hello
```

### Standard error (stderr)

Is an error message outputted by a failed process.

[Back to index](#)

```
PS C:\Users\maria\documents> ls documents
ls : Cannot find path 'C:\Users\maria\documents\documents' because it does not exist.
```



## 2. Location and navigation

### Print Working Directory

It helps to identify where in your file system your shell is currently located at. It gives the **absolute path**.

[Back to index](#)

`pwd`

### List

Lists all files and folders at your current location except the hidden ones (preceded by a dot).

[Back to index](#)

`ls`

### List -a

Lists all files and folders at your current location including the hidden ones.

[Back to index](#)

`ls -a`

### List -l

Lists all files and folders in a long list format. Apart from listing the contents, it also gives information about each file and folder, like the author, the date it was created, etc.

[Back to index](#)

`ls -l`

### List -la

It concatenates the -l and -a flags to list all files, including the hidden ones, in a long list format.

[Back to index](#)

`ls -la`



## Change Directory

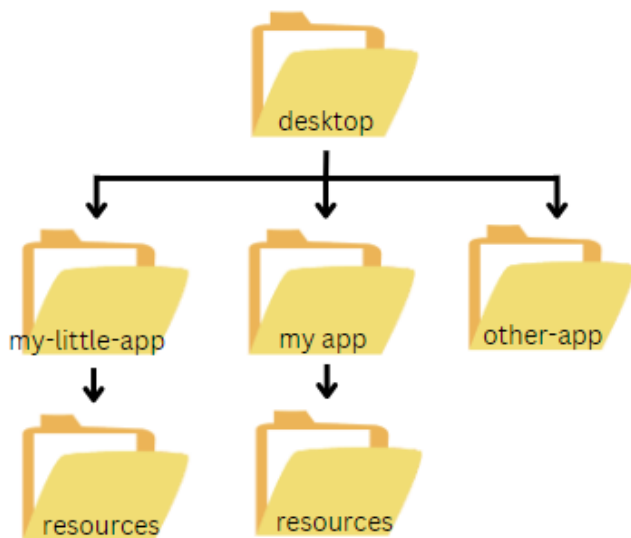
Switches the location of the terminal from the current working directory to the root directory of the computer.

[Back to index](#)

## Change Directory <relative path>

Switches the location of the terminal from the working directory to the directory defined by the **relative path**.

This relative path expresses the location of the desired directory in relation to the working directory. On the right side there are some examples using the following file structure:



[Back to index](#)

`cd`

## Desktop as Working directory

`cd my-little-app` -> switches the working directory from desktop to its child, my-little-app

`cd my-little-app/resources` -> switches the working directory from desktop to its grandchild, resources (child of my-little-app)

`cd ..` -> switches the working directory from desktop to its parent.

## My-little-app as Working directory

`cd ../"my app"` -> switches the working directory from my-little-app to its sibling, my app

`cd ../"my app"/resources` -> switches the working directory from my-little-app to its sibling's child, resources (child of my app)

`cd ../../` -> switches the working directory from resources to desktop





### 3. Managing files and folders

#### touch

Creates a file with the name and extension passed in the command.

[Back to index](#)

```
touch index.html
```

#### mkdir

Creates a new folder with the name passed in the command.

[Back to index](#)

```
mkdir my-little-app
```

#### echo

Accepts a string of standard input and echoes the string as standard output. If no file is specified, the string will be echoed to the terminal.

[Back to index](#)

```
echo "hello"
```

#### cat

Outputs the content of a specific file into the terminal.

[Back to index](#)

```
cat hello.txt
```

#### cp

Copies a source file/directory or the content of a source file/directory into a destination file/directory. It takes at least two arguments, first the file/directory to be copied and then the file/directory where the copy is going to be made.

[Back to index](#)

```
cp source.txt destination.txt -> copies the content of source.txt into destination.txt
```

```
cp source.txt destination -> creates a copy of the source.txt in the destination directory
```

```
cp source.txt destination/new-name.txt -> creates a copy of the source.txt in the destination directory and renames it to new-name.txt
```

```
cp source1.txt source2.txt destination -> creates a copy of the source1.txt and source2.txt in the destination directory
```



## mv

Moves a file in a similar way that cp does, but without creating a copy. It takes as a first argument the file to be moved (or a list of files), and as the second argument the directory where it will be moved.

It can also be used to rename a file by passing the name of two files.

[Back to index](#)

**mv source.txt destination/** -> moves the source.txt to the destination directory

**mv source1.txt source2.txt destination/** -> moves source1.txt and source2.txt in the destination directory

**mv original.txt renamed.txt** -> renames source.txt into destination.txt

## rm

Deletes the file with the name passed in the command

[Back to index](#)

**rm index.html**

## rm -r

The flag -r (recursive) is used to delete a directory and all its child directories and files.

[Back to index](#)

**rm -r my-little-app**

## rm -f

The flag -f (force) is used to force the operating system to delete the file, even if it's locked or protected.

[Back to index](#)

**rm -f my-little-app**

## rm -rf

The flags -rf (recursive, force) are used to forcefully and recursively remove files and directories.

[Back to index](#)

**rm -rf my-little-app**



## rmdir

Deletes the folder with the name passed in the command. It won't work unless it is empty.

[Back to index](#)

## wildcards

Special characters used to select groups within a command. They can be used with suffixes and prefixes to specify the pattern of files to select.

[Back to index](#)

`rmdir my-little-app`

`cp * destination/` -> creates a copy of all the files from the working directory into the destination directory

`cp *.txt destination/` -> creates a copy of all the .txt files from the working directory into the destination directory

`cp s*.txt destination/` -> creates a copy of all the .txt files that start with an s from the working directory into the destination directory



## 4. Concatenate commands

### Concatenating commands ;

Using the semicolon between commands will concatenate them so you can write several commands at once.

[Back to index](#)

```
mkdir my-little-app; cd my-little-app; touch index.html;  
mkdir resources; ls my-little-app;
```



# CHEAT SHEET

Web & App Development



## 5. CLIs

### Git

It enables git commands if the software is installed.

[Back to index](#)

git **init**

### Node

It enables the commands from the “Node Package Manager” if Node.js is installed on your computer.

[Back to index](#)

node **-v**

### NPM

It enables the commands from the “Node Package Manager” if Node.js is installed in your computer.

[Back to index](#)

npm **install**



## 6. Redirecting input and output

>

Redirects standard output of a command to a file, overwriting previous content.

[Back to index](#)

```
echo "hello" > about.html  
cat index.html > about.html
```

>>

Redirects standard output of a command to a file, appending new content to the previous content.

[Back to index](#)

```
echo "hello" >> about.html  
cat index.html >> about.html
```

<

Redirects standard input to a command

[Back to index](#)

```
cat < index.html
```

|

It works as a pipe, redirecting the standard output from the previous command as the standard input to the next command.

[Back to index](#)

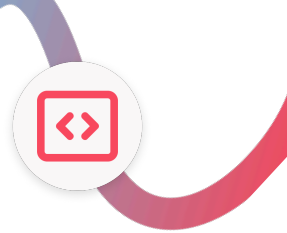
```
cat hello.txt | wc  
cat hello.html | wc | cat > index.html
```

### Sort

Sorts the standard input alphabetically and outputs it to the terminal. It doesn't change the file itself.

[Back to index](#)

```
sort names.txt  
cat greetings.txt | sort > sorted-greetings.txt
```



## Uniq

Filters duplicate, adjacent lines of the standard input to the standard output.

[Back to index](#)

```
uniq names.txt
```

```
sort names.txt | uniq
```

```
sort names.txt | uniq > uniq-names.txt
```

## Grep

Searches for a text pattern within the input and outputs it. It is case sensitive.

[Back to index](#)

```
grep Lakes bodies-of-water.txt
```

## Grep -i

The flag -i (insensitive) makes the grep command case insensitive.

[Back to index](#)

```
grep -i Lakes bodies-of-water.txt
```

## Grep -R

The flag -R (recursive) searches for a text pattern within all the files of a specified directory and outputs filenames and lines with the matching result.

[Back to index](#)

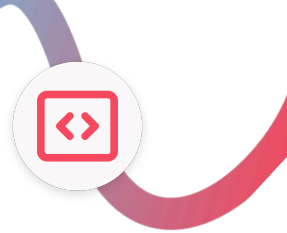
```
grep -R Lakes geography
```

## Grep -RI

The flag -RI (recursive, files with matches) searches for a text pattern within all the files of a specified directory and outputs only the filenames with matching results.

[Back to index](#)

```
grep -RI Lakes geography
```



## Sed

Searches for a text pattern within the input, modifies it based on an expression, and outputs it. It doesn't rewrite the file with the output.

The expression is the first parameter. It is written between quotation marks and build with:

- **Line number** -> when added, it will specify in which line to apply the command.
- **Range of lines** -> when passed, it will specify between which lines the command will be applied. The \$ indicates the last line.
- **s** -> stands for substitution and it is always used when using sed for substitution.
- **Search string** -> the string or text to find and substitute.
- **Replacement string** -> string or text to be added in place of the search string.
- **g** -> stands for global, and it is used when we want all the instances of the search string within a line to be substituted for the replacement string. Without this, only the first instance of a line that matches the search string.
- **d** -> stands for delete and it is added to delete a line, a range of lines or a line with a specific pattern.

The file to search for a pattern is the second argument.

[Back to index](#)

## Sed -i

The flag -i makes it so the file is rewritten with the output of the sed command.

[Back to index](#)

## Substitute

`sed "3 s/lake/river" bodies-of-water.html`

`sed "2,6 s/lake/river" bodies-of-water.html`

`sed "s/lake/river" bodies-of-water.html`

`sed "s/lake/river/g" bodies-of-water.html`

## Delete

`sed "/lake/d" bodies-of-water.html` -> deletes lines where lake is written in.

`sed "3d" bodies-of-water.html` -> deletes line number 3

`sed "$d" bodies-of-water.html` -> deletes the last line

`sed "6,$d" bodies-of-water.html` -> deletes the range of lines between line 6 and the last line

`sed -i "s/lake/river/g" bodies-of-water.html`





## 7. Nano editor

### Nano

It will open a file in the nano command line text editor. The editor will allow you to modify the content of files within the terminal.

[Back to index](#)

`nano hello.txt`

### Saving changings

Once the file content has been modified, the command on the right will allow the changes to be saved.

`Ctrl + O`

The nano editor will ask in which file to save the information, giving the name of the file opened in the nano as default. Press enter to finish saving the

[Back to index](#)

`Enter`

### Copy

Sets the beginning of the target text to be copied. This allows the cursor to select the text from that point till the end of the text to be copied by moving it with the arrow keys.

`Ctrl + 6`

To copy the selected text.

[Back to index](#)

`Alt + 6`



## Paste

Pastes the text where the cursor is in the file.

[Back to index](#)

Ctrl + U

## Undo

To undo a change.

[Back to index](#)

Alt + U

## Redo

To redo a change.

[Back to index](#)

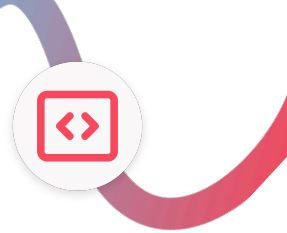
Alt + E

## Exiting nano

To get out of the nano editor and back to the terminal.

[Back to index](#)

Ctrl + X



## 8. Configuring the command line environment

### Environment

It refers to the settings and preferences of the current user.

[Back to index](#)

```
sed -i "s/lake/river/g" bodies-of-water.html
```

### Bash profile

File used to store environment settings for the terminal or command line, such as aliases, variables and commands;

Every time a new session is started in the terminal, the `.bash_profile` is loaded so all the configurations in it will be available.

The name uses the `.` to indicate is a hidden file.

[Back to index](#)

```
.bash_profile
```

### Source

Activates the changes done to the `.bash_profile` without having to restart the terminal.

[Back to index](#)

```
source .bash_profile
```

### Alias

Allows for a shortcut to be created for a command. That way, instead of typing the command, you can type the alias in the terminal instead.

[Back to index](#)

```
alias c="clear"
```

```
alias ll="ls -la"
```

### Environment Variables

They are variables that hold information about the environment and can be used in commands.

[Back to index](#)

```
export USER="Jane Doe"
```



## PS1

Variable that holds the information on how to style the command line prompt. Add it to the `.bash_profile` to change the default styling.

[Back to index](#)

```
export PS1=">> "
```

## HOME

Variable that stores the path to the home directory. It can be specified or changed in the `.bash_profile`, but is not needed in most cases.

Use the `echo` command to return its value.

[Back to index](#)

```
echo HOME
```

## PATH

Variable that stores a list of directories separated by a colon. The directories stored there are the ones that hold the scripts for the command line to execute.

[Back to index](#)

```
echo PATH
```

## env

Stands for environmental and is a command that returns a list of all the variables set for the current user.

[Back to index](#)

```
env
```